

COVER_RANGE

Overview

cover_range collects a range coverage for the multi-bit data input. Users may define any number of ranges and the minimum & maximum values for each of the predefined ranges. When ALL counters reach their minimum limits, the **min_limit** access event toggles. When ONE of counters reaches its maximum limit, the **max_limit** access event toggles. All coverage information can be accessed dynamically in order to tune test the generation process and define additional constraints and length of the simulation run.

Syntax

```
cover_range #(bit_width,cover_num, ssize) InstName (clk, rst_n, data);
```

Parameters		
Bit_width	Bit width of the data input	
cover_num	Maximum number of coverage items allowed. Default: 0	
Ssize	String size for the name of the cover item. Default: 20	
Interface		
clk	Sampling clock for data	
rst_n	Active-low reset	
data	Input data signal, for coverage collection. Bit width of this signal has to be defined using "bit_width" parameter.	
Access Tasks		
Name	Arguments	Description
reset	No	Resets coverage counters and maximum & minimum limits
add	cover_name start_range end_range min_limit max_limit	- name of the coverage Item (with length <= ssize) - Beginning of range - End of range - Lower limit for the counter. Default: 0 - Upper limit for the counter. Default: 32'hFFFF_FFFF Use "add" access task to add functional Items to the cover_range coverage group. Example: <InstName>.add(("RNG 1", 10, 20, 5, 8); – add coverage item for range "RNG 1", lower limit eq. to 5and upper limit equals to 8
change	cover_name start_range end_range count min_limit max_limit	- name of the coverage Item (with length <= ssize) - Beginning of range - End of range - Counter number for the coverage item - Lower limit for the counter. Default: 0 - Upper limit for the counter. Default: 32'hFFFF_FFFF Use "change" access task to edit functional Item data. Example: <InstName>.edit(("SEG 1", 10, 20, 0, 5, 10) – resets counter and change max. limit from 8 to 10 for the coverage range "SEG 1"
report	No	Generates coverage report in the tabular format. See below for the report example.
Access Events		
covered	"Fires" when all vc_cover counters exceed their lower limits. Can be monitored externally to define length of the test case execution based on the accumulated coverage information.	
exceeded	"Fires" when any of the counters exceed a predefined maximum limit. The maximum limits can be used as additional test-specific constraints. When it fires, "Exceeded_name" access register provides the name of the cover item exceeding the maximum limit.	

Access Variables	
total_cov	Provided a constantly updated value of the total coverage (in %). After “covered” event, it has to be equal to 100 %.
exceeded_name	Name of the coverage item with the counter exceeding a predefined maximum value.
Control Variables	
Display_status	Set it to “1” to get more informational messages. Default: 0

Verilog example

The example below shows how coverage information controls the simulation run. By monitoring a “covered” event, the test case is running until a defined coverage is accumulated. By monitoring an “exceeded” event, the test case exits upon the first violation.

```

module test;

reg clk; initial clk = 0;
reg [6:0] data;

parameter INIT = 0;
parameter S1 = 1;
parameter S2 = 2;
parameter S3 = 3;
parameter S4 = 4;
parameter S5 = 5;
parameter S6 = 6;
parameter S7 = 7;

always #10 clk <= ~clk;

always @(posedge clk)
    data <= ($random)%100;

//-----
cover_range #(7,5) cov (clk, 1'b1, data);

initial begin
    cov.display_status = 0;
    //      name      from to min max
    //      -----
    cov.add("RANGE 1", 0, 19, 5, 100);
    cov.add("RANGE 2", 20, 39, 5, 100);
    cov.add("RANGE 3", 40, 59, 5, 100);
    cov.add("RANGE 4", 60, 79, 5, 100);
    cov.add("RANGE 5", 80, 99, 5, 100);
end
//-----

initial forever begin
    #1000 cov.report;
end

// Exit on first maximum violation
always @(cov.exceeded) begin
    $display ("[%0t] TC INFO: MAXIMUM reached for '%s'",
        $time, cov.exceeded_name);
    $finish;
end

```

```

// Run till all functional coverage is collected; then exit
always @(cov.covered) begin
    $display("[%0t] TC INFO: Stimulus coverage reached, test may exit", $time);
    cov.report;
    $finish;
end

//If functional coverage is not collected , use timebomb
initial begin
    $dumpvars;
    #10000 $finish;
end

```

Range Coverage Report Example

Range Coverage Report <test.cov.report>					

Cover name	Count	Min_limit	Max_limit	Coverage	

RANGE 1	3	5	100	60	percent
RANGE 2	6	5	100	100	percent
RANGE 3	11	5	100	100	percent
RANGE 4	6	5	100	100	percent
RANGE 5	13	5	100	100	percent
Total Coverage:		92 percent			